

DPF Sentinel — Easy Setup Guide

Easy setup — no technical experience needed

This page is for someone who has **never used a terminal, never installed developer tools, and has no idea what “flashing firmware” means**, but wants to build their own DPF monitor for a 2014 Hyundai ix35 1.7 CRDi. Everything here is point-and-click. If you’re comfortable with a command line already, [manual-setup.md](#) and [workstation-setup.md](#) are faster.

This guide builds **one board: the CYD 3.5” touchscreen** (`boards/cyd-s3-3p5` in this repo). It’s the right one to start with — no soldering, no Raspberry Pi, no Linux skills, and it doesn’t need your phone once it’s set up.

Two ways to do this

Option A — have an AI assistant walk you through it. Open [Claude](#) or [ChatGPT](#) in a browser tab, and paste the box below. It will ask you questions and give you one step at a time, in plain language, checking in before anything that matters.

```
I'm a complete beginner and I want to build a DPF (diesel particulate filter) monitor for my car from an open-source project, using a ready-made ESP32-S3 touchscreen board called "CYD 3.5-inch". I have no programming or command-line experience. Walk me through this from scratch, GUI only – no terminal commands unless there is truly no other way, and if there isn't, explain what the command does in plain English before I type it.
```

```
The steps, in order, are documented at:  
https://github.com/vitorhpsousa/DPFSentinel/blob/main/docs/easy-setup/README.md (this page – the shopping list and the config file walkthrough) and  
https://github.com/vitorhpsousa/DPFSentinel/blob/main/docs/easy-setup/windows.md (or mac.md / linux.md, whichever matches my computer) for installing the software
```

and
flashing the board. Read those pages first – ask me to paste them if
you can't
open the link. Then ask me which operating system I'm on and follow
only that
page.

Rules for you:

- One step at a time. After each step, tell me exactly what I should
see if it
worked, and wait for me to confirm before the next one.
- Never ask me to paste my WiFi password, Telegram bot token, or any
other
secret into this chat. Tell me which file to put it in on my own
computer
instead, and exactly what line to change.
- If something doesn't look like the guide says, stop and help me
figure out
why rather than telling me to skip it.
- Tell me plainly if something I'm asking for isn't possible or isn't
a good
idea, rather than guessing.

I'll tell you when I'm ready for step one.

Option B — do it yourself, no AI. Follow the written steps below, then pick your
operating system:

- [Windows](#)
- [Mac](#)
- [Linux](#)

Both options use the exact same steps underneath — Option A just has someone
explaining each one to you as you go.

What you're building

A small touchscreen box that plugs into your car's OBD-II port (the diagnostic
socket, usually under the dashboard near the steering column — the same one a
garage's scanner plugs into) via a small Bluetooth adapter. It reads how much soot is
in your diesel particulate filter and warns you before it clogs — it doesn't change
anything about how your car runs, it only reads data the engine computer already
has. It does **not** need your phone once it's working, and it does **not** need you to pair
anything in your computer's or phone's Bluetooth settings — the board talks to the
adapter directly, on its own.

Shopping list

1. **The CYD 3.5" board** — sold on Amazon/AliExpress as “CYD ESP32 S3 Display 3.5 Inch IPS Capacitive Touch 320x480 ST77922 HMI Module, XiaoZhiAI” (around £20–25). A USB-C cable comes with most listings, but make sure you have one — see the warning below. If you’re tempted by a different/cheaper board instead: avoid any board whose only antenna is a small ceramic chip part with no external connector (no U.FL/IPEX plug, no printed antenna trace on the board itself) — three separate boards of that type were tried in this project and none could hold a steady Bluetooth connection to the OBD adapter once mounted in the car. The CYD board above doesn’t have this problem.
2. **A Bluetooth OBD-II adapter.** It must be **BLE** (“Bluetooth Low Energy” / “Bluetooth 4.0+”), not “Bluetooth Classic” — the board can only talk to the BLE kind. Look for one that says BLE or “Bluetooth 4.0/5.0” in its listing. Adapters from **Veepeak** or **Konnwei** that advertise BLE support are known to work with this project; a generic one whose listing mentions “BLE” should also work, since the board searches for anything with “OBD”, “ELM”, “VEEPEAK”, “VLINK”, “OBDLINK” or “KONNWEI” in its name. Around £15–20.
3. **A microSD card**, 32 GB or smaller, plus a card reader for your computer if it doesn’t have a slot built in (many laptops don’t — a USB card reader is a couple of pounds). **Larger than 32 GB will not work** without an extra step — see the FAT32 warning on your OS page.
4. **A USB cable that actually carries data, not just power.** Some cheap cables only charge and cannot transfer files — if your computer never sees the board after plugging it in, this is the first thing to try swapping.
5. Something to power the board once it’s living in the car — a spare phone charger plugged into a 12V-to-USB adapter, or your car’s own USB port if it has one, both work fine.

Setting up your own secrets (do this after installing the software)

Every board needs to know your WiFi network name and password before it can connect to the internet, and — if you want phone alerts — a Telegram bot token. These live in **one file on your own computer**, never typed into a chat with anyone, AI or human.

1. In the project folder, go to `boards/cyd-s3-3p5/firmware/src/`. You’ll see a file called `config.example.h`. **Make a copy of it in the same folder and rename the copy to `config.h`** (not `config.example.h` — the board only reads `config.h`).
2. Open `config.h` in a plain text editor (VS Code, which you’ll already have installed by this point, works fine — double-click the file).

- Find these two lines and put your own WiFi network name and password between the quote marks, replacing the placeholder text exactly:

```
#define WIFI_STA_SSID "Vitoi"
#define WIFI_STA_PASS "YOUR_WIFI_PASSWORD"
```

- Optional — Telegram alerts on your phone.** If you want a message when a regeneration starts or finishes, or a daily report:

- Open Telegram, search for **@BotFather**, and send it `/newbot`. Follow its prompts (it asks for a name, then a username ending in `bot`). It replies with a long token that looks like

```
123456789:AAFxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx — copy it.
```

- Send your new bot any message (e.g. “hi”) so it knows who you are.
- In a browser, go to `https://api.telegram.org/bot<your token>/getUpdates` (with your real token in place of `<your token>`). Look for `"chat":{"id":` in the page that loads — the number right after it is your chat ID.
- Back in `config.h`, fill in:

```
#define TG_BOT_TOKEN "your token here"
#define TG_CHAT_ID "your chat id here"
```

- Don't want Telegram? Leave both lines exactly as they are (`YOUR_TELEGRAM_BOT_TOKEN` / `YOUR_CHAT_ID`) — the feature switches itself off when the token is empty-looking like that.
- Leave every other line in `config.h` as it is unless your OS-specific page tells you otherwise. **Never share your `config.h` file, and never commit it or paste its contents anywhere** — it's listed in this project's `.gitignore` specifically so a normal “save/commit my changes” step can't accidentally publish it.

Formatting the microSD card

The board needs the card formatted as **FAT32**. This matters because if the card is blank, unformatted, or in the wrong format, the board **will not warn you** — it silently falls back to its own tiny internal memory (room for a few minutes of driving) instead. Each OS page below has the exact formatting steps for that operating system — do this before the card's first use.

Checking it's actually working

After you've flashed the board (your OS page covers this) and inserted the formatted SD card:

1. Power the board from USB. You should hear a short boot jingle and the screen should light up.
2. With no car connected yet, the screen shows **"NO LOGGER DATA"** — that's expected; it means the board itself is alive but hasn't found an OBD adapter yet.
3. Plug your Bluetooth OBD adapter into the car and turn the ignition to "on" (engine doesn't need to be running). Within a few seconds to a minute, the screen should change to show live numbers on a coloured background:
 - **Green** — everything normal, soot is low.
 - **Amber** — soot is building up, keep an eye on it.
 - **Red** — soot is close to the point where the car will want to regenerate (burn it off) on its own.
 - **A dark screen saying "ACTIVE REGENERATION"** — it's burning the soot off right now; the screen tells you not to switch the engine off until it finishes.
4. If you set up WiFi, the board also joins your home network and your phone can see a live dashboard. Each OS page shows how to find its address.

If none of that happens, each OS page has a troubleshooting section for the most common first-time issues.

A note on safety

This is a hobby project, not a diagnostic tool — it reads numbers the car's engine computer already has and cannot tell you the filter is actually blocked or safe. See [disclaimer.md](#) for the full version. Don't interact with the screen or your phone while driving.

A note on privacy

This device never needs an internet connection to work, and nobody out on the wider internet can find it or control it — there's no way in from outside your own WiFi.

What it *does* do: it makes its own small WiFi hotspot (like your home router does) so your phone can open its live-data web page, and it can also join your home WiFi so you can check on it from the house. Either way, **being on that WiFi is the only way in** — and the live-data page itself isn't password protected. So:

- **Treat the device's hotspot password like your home WiFi password.** Anyone who knows it and connects to it could see your driving data while connected — nothing more, they can't damage the car or the device.

- **The same is true while it's joined to your home WiFi:** anyone already on your home network could open the same page.
- This is normal for small home-gadget projects like this one, not a mistake in the setup. Just don't hand the hotspot password to strangers, the same way you wouldn't hand out your home WiFi password.

The Telegram alerts are separate and safe: they go over a normal, secure connection straight to your own phone's chat with the bot, the same way any messaging app works. Nobody else can read those messages, and the device only ever listens to your own chat — anyone else messaging the bot is silently ignored.

Not ready to build one yourself?

If all of this still sounds like too much, that's completely fine — ask whoever gave you this link about getting a board that's already set up and tested, ready to just plug in.

Found this useful?

This project is free and always will be. If it saved you some hassle and you'd like to say thanks, ko-fi.com/vitoi — no pressure either way.

Windows — install the software and flash the board

New to this? Start at the [easy-setup hub page](#) for the shopping list and the config walkthrough.

This page covers two things only: getting the tools installed on your Windows computer, and getting the firmware onto the CYD 3.5” board. It is 100% point-and-click — there is no command line in this page.

Everything below has a specific place in the order. Do them in order; later steps (like the config file) assume earlier ones (like having the project folder) are already done.

1. Install Visual Studio Code

VS Code is the program you’ll use for everything else on this page — editing the config file, building the firmware, and sending it to the board.

1. Go to code.visualstudio.com in your browser.
2. Click the big **Download for Windows** button.
3. Once it’s downloaded, double-click the installer file (it will be in your **Downloads** folder, named something like `VSCodeUserSetup-x64-...exe`).
4. Click through the installer: accept the licence agreement, then click **Next** on every screen without changing anything. The default options are fine.
5. Click **Install**, then **Finish**. VS Code should open on its own.

2. Install the PlatformIO extension

PlatformIO is the piece that understands how to build this project and talk to the board. It installs as an extension inside VS Code — you don’t download it separately.

1. In VS Code, look at the row of icons down the far left edge of the window. Click the one that looks like four squares (the **Extensions** icon).
2. A search box appears at the top of the panel that opens. Type **PlatformIO IDE**.
3. Click on **PlatformIO IDE** in the results (it’s normally the top result, published by “PlatformIO”), then click the blue **Install** button.
4. Wait. This downloads and sets up a fair amount of software in the background — it can take **several minutes** the first time, even on a fast connection. You’ll see progress notifications in the bottom-right corner of the window.
5. VS Code may ask to **reload** the window partway through, or do it on its own. Let it. When it’s done, you should see a new icon that looks like an alien head (the PlatformIO icon) added to that same left-hand strip of icons.

3. Get the project files

You need the `obd-esp32` project on your own computer. Which of these two you do depends on how you were given the project:

If you have a repository URL (e.g. a Gitea or GitHub-style link someone gave you):

1. In VS Code's left-hand icon strip, click the PlatformIO (alien head) icon.
2. In the PlatformIO Home tab that opens, look for a **Clone Git Project** (or similar wording) option, or use VS Code's own built-in Git support: press `Ctrl+Shift+P`, type **Git: Clone**, press Enter, then paste the URL you were given and press Enter again.
3. When asked, pick a folder to save it into — anywhere you'll remember, such as your **Documents** folder.
4. When it finishes, VS Code will likely offer to open the folder it just downloaded. Don't open it yet — the folder you actually need to open in the next step is a subfolder, not this top-level one.

If you were given a folder or a ZIP file directly (e.g. on a USB stick or downloaded link):

1. If it's a ZIP file, right-click it in File Explorer and choose **Extract All...**, then choose a destination such as your **Documents** folder and click **Extract**.
2. You should end up with a folder named something like `obd-esp32` containing files like `platformio.ini`, a `boards` folder, and so on.

Either way, note where this `obd-esp32` folder ends up — you'll need to find it again in the next step.

4. Open the correct project folder in PlatformIO

This repository holds the firmware for **several different boards**, not just one. Each board's own PlatformIO project lives in its own subfolder, and PlatformIO needs you to open that specific subfolder as the project — not the repository's top-level folder — or it won't know which board you mean.

For the CYD 3.5" board, the folder you need is:

```
obd-esp32/boards/cyd-s3-3p5/firmware
```

To open it:

1. In VS Code, go to the **File** menu (top-left) and click **Open Folder...**
2. Browse to wherever you put the `obd-esp32` project, then go into `boards`, then `cyd-s3-3p5`, then `firmware`. Select that `firmware` folder and click **Select Folder**.

3. VS Code will reload with that folder open. You should see a left-hand file list including `platformio.ini`, a `src` folder, and so on — if you instead see a long list of other boards' folders, you opened the wrong level; go back and open `firmware` specifically.
4. PlatformIO will notice the project and may spend a minute or two in the background preparing it the first time (downloading the specific tool versions this project needs). You can watch this in the bottom-right notifications.

A note for later: this project's `platformio.ini` deliberately pins an older tool version (`platform = espressif32@7.1.3`) because a newer one is known to crash this firmware's Bluetooth code. If PlatformIO or VS Code ever offers to **update** this project's platform or toolchain, decline it — leave that line exactly as it is.

5. Set up your WiFi and Telegram secrets

Now that you have the project open, do the `config.h` step before building anything. This is already fully explained on the hub page — see [Setting up your own secrets](#) on the [easy-setup hub page](#). Come back here once `config.h` is filled in.

6. Format the microSD card as FAT32

Do this before you power the board for the first time — an unformatted or wrongly-formatted card fails silently (see the hub page's warning).

1. Put the microSD card into your computer's card slot, or into a USB card reader if your computer doesn't have one built in.
2. Open **File Explorer** (the folder icon in your taskbar) and find the card under **This PC** — it'll show up as a drive letter like `D:` or `E:`.
3. Right-click that drive and choose **Format....**
4. In the **File system** dropdown, choose **FAT32**. (It may already be selected by default on a small card.)
5. Leave **Allocation unit size** on its default, and click **Start**. Click **OK** on the warning that this erases the card — that's expected for a new or repurposed card.
6. When it finishes, close the dialog and insert the card into the board.

If your card is larger than 32 GB, Windows' Format dialog will refuse to offer FAT32 at all — this is a built-in limitation of Windows' own formatter, not a mistake on your part. Your options are: use a 32 GB or smaller card instead (the simplest fix), or use a third-party FAT32 formatting tool that supports larger cards. This project doesn't need a large card, so switching to a smaller one is the easiest route.

7. Connect the board over USB

1. Plug the board into your computer using a USB-C cable. Make sure it's a cable that actually carries data, not just power — the shopping list on the hub page has a note about this; a charge-only cable will power the screen on but your computer will never see it as a device.
2. Windows should recognise it within a few seconds. You might see a small notification bottom-right saying a device was set up, or nothing visible at all — both are normal.
3. This board has no separate USB-to-serial chip (no CP2102, no CH340/CH9102 — if you've seen generic ESP32 guides telling you to install one of those drivers, that advice doesn't apply to this board). It talks to your computer through the ESP32-S3 chip's own built-in USB connection, which Windows 10 and 11 normally recognise automatically with no extra driver install. **This specific claim is unverified on a genuinely fresh Windows machine** — it's how this type of connection is supposed to behave, but it hasn't been personally confirmed for this exact board model start-to-finish.
4. To check it worked, open **Device Manager** (right-click the Start button, choose **Device Manager**) and look under **Ports (COM & LPT)** for a new entry like `COM3` or similar — note the number if you need it later (you usually won't; PlatformIO finds it automatically).

If no COM port shows up, see the troubleshooting table below before continuing.

8. Build and upload the firmware

The controls you need live in the **blue toolbar at the very bottom of the VS Code window** (not the top) once PlatformIO has finished preparing the project.

1. Look at the bottom status bar. You should see a row of small icons, including a **checkmark (✓)** and a **right-pointing arrow (→)**.
2. Click the **checkmark icon** first — this is **Build**. It compiles the firmware without sending it to the board yet, which is a good first check that everything is set up correctly.
3. A panel opens at the bottom of the window (the **terminal pane**) showing progress text scrolling past. **The first build can take several minutes** — it's downloading and compiling a lot of underlying code the first time. Later builds are much faster.
4. When it finishes, look for a line in green text saying `[SUCCESS]`. If instead you see red text and the word `[FAILED]` or `Error`, something needs fixing before continuing — scroll up in that same pane to read the actual error message.
5. Once the build succeeds, click the **right-arrow icon (→)** — this is **Upload**. It builds again (reusing most of the previous work, so it's faster) and then sends the firmware to the board over USB.

6. Watch the same terminal pane. A successful upload also ends with a green **[SUCCESS]** line, after some output about connecting to the board and writing data to it. The board may flicker or restart during this — that's normal.

9. Open the Serial Monitor

The Serial Monitor is a window that shows you text the board itself is printing as it runs — its own internal commentary on what it's doing. This is how you confirm it booted properly without just guessing from the screen.

1. In the same bottom toolbar, find the icon that looks like an electrical **plug** — this is the Serial Monitor.
2. Click it. A new panel opens showing live text from the board.
3. Power-cycle the board (unplug and replug the USB cable, or just wait if it already restarted from the upload) and watch the text scroll by. You're looking for lines mentioning booting up, the screen jingle happening, and a Bluetooth/BLE scan starting — confirming the firmware is alive and looking for your OBD adapter.
4. You can close this panel once you've seen it boot successfully.

10. Troubleshooting (Windows-specific)

Problem	Likely cause	What to try
No COM port appears in Device Manager	Charge-only USB cable, or the board isn't getting power	Swap to a different USB cable that you know carries data (not just power), and try a different USB port on your computer.
COM port appears but shows a yellow warning icon, or says "Unknown device"	Windows couldn't match it to a driver automatically (unverified how often this happens on this board)	Try unplugging and replugging first. If it persists, check for Windows Updates (Settings > Windows Update > Check for updates), since USB-CDC driver support sometimes ships that way. As a last resort, search the board listing/manufacturer page for any driver notes.

Problem	Likely cause	What to try
Upload fails with “Access is denied” or the port is busy	Another program has the COM port open	Close any other serial monitor, Arduino IDE window, or a second VS Code window that might also have PlatformIO’s monitor open on the same port, then try Upload again.
Upload starts but fails partway through	Loose or poor-quality USB cable, or something interrupted the connection (closing a laptop lid, a USB hub losing power)	Try a shorter or known-good cable, plug directly into the computer rather than through a USB hub, and don’t touch the cable during upload.
PlatformIO extension install hangs or fails	A patchy internet connection during the first-time download, or a work/school network blocking the download	Try again on a stable connection; check the notification area for a specific error message. (unverified) which proxies or filtered networks specifically cause this.
Upload fails and antivirus/Windows Defender flagged something	Some antivirus tools are known to interfere with flashing tools like esptool because they briefly write to the port in an unusual way (unverified for this specific project)	Temporarily pause real-time protection for the upload only, then re-enable it afterwards. Don’t leave it disabled.
Build fails right after opening the folder, mentioning a missing platform or package	PlatformIO is still downloading the pinned toolchain in the background	Wait for the bottom-right notifications to finish, then try Build again.

What’s next

Once the build and upload both say `[SUCCESS]` and the serial monitor showed a clean boot, go back to the hub page’s [Checking it’s actually working](#) section — it walks through what you should see on the board’s own screen once it’s plugged into the car.

Easy setup — Mac

New to this? Start at the [easy-setup hub page](#) for the shopping list and the config walkthrough.

This page covers two things only: installing the software on your Mac, and getting the firmware onto the CYD 3.5” board. It assumes you’ve already read the hub page and have the board, a data-capable USB-C cable, and (ideally) a 32 GB or smaller microSD card in hand.

Everything below is clicking and typing into windows — no Terminal, except one optional aside near the end that you can skip entirely.

1. Install Visual Studio Code

VS Code is the program you’ll use to write the firmware onto the board. It’s free, made by Microsoft, and has nothing to do with the actual car-reading code — it’s just the editor/toolbox.

1. Open your web browser and go to **code.visualstudio.com**.
2. Click the big **Download for macOS** button (the site detects you’re on a Mac and offers the right file automatically).
3. A file ending in **.zip** downloads. Click it in your browser’s download list, or open your **Downloads** folder and double-click it — macOS will unzip it automatically into a file called **Visual Studio Code.app**.
4. Drag **Visual Studio Code.app** onto your **Applications** folder (you can open Applications from Finder’s sidebar, then drag the icon across).
5. Open **Applications** and double-click **Visual Studio Code**.
6. The first time you do this, macOS will show a warning like *“Visual Studio Code” is an app downloaded from the Internet. Are you sure you want to open it?* — this is normal (it’s called Gatekeeper, and it shows up for almost any app you download outside the App Store, not just this one). Click **Open**.

VS Code should now launch and show an empty window with a welcome screen.

2. Install the PlatformIO extension

PlatformIO is the piece that actually knows how to build the firmware and send it to the board. It installs as an extension *inside* VS Code — you don’t download it separately.

1. In VS Code, look at the strip of icons down the left edge. Click the one that looks like four squares (the **Extensions** icon).
2. A search box appears at the top of the panel. Type **PlatformIO IDE**.

3. The official result is called **PlatformIO IDE**, published by PlatformIO. Click **Install** on it.
4. Wait. The first install downloads quite a lot (compilers, tools for every chip PlatformIO supports) and **can take several minutes**, even on a good connection. Don't close VS Code while it's working.
5. When it finishes, VS Code will likely ask to **reload** the window, or just reload itself. Let it. After reloading, you should see a new icon in the left-edge strip that looks like an alien head/ant — that's the PlatformIO icon, and it means the install worked.

3. Get the project files

You need the `obd-esp32` project folder on your Mac. Depending on how you were given access to it, use whichever of these applies:

If you have a repository URL (a Git address, e.g. something ending in `.git`, from a Gitea or GitHub-style server):

1. Click the PlatformIO icon in the left strip, then look for **Open** or a **Clone** option in the PlatformIO home screen — PlatformIO's home tab has a **"Clone Git Project"** button. Click it.
2. Paste the URL you were given, choose a folder on your Mac to save it into (your **Documents** folder is a sensible choice), and confirm.

If you were given a folder or a `.zip` file directly (e.g. a USB stick, an email attachment, a shared drive):

1. If it's a `.zip` file, double-click it — macOS extracts it automatically into a regular folder with the same name, in the same location.
2. Move that extracted folder somewhere easy to find, such as your **Documents** folder.

Either way, you should end up with a folder named something like `obd-esp32` somewhere on your Mac, containing (among other things) a `boards` folder.

4. Set up your secrets

Before building anything, go do the `config.h` step now: copy `config.example.h` to `config.h` inside `boards/cyd-s3-3p5/firmware/src/` and fill in your WiFi (and optionally Telegram) details. This is fully explained in the [hub page's "Setting up your own secrets" section](#) — do it now, then come back here.

5. Open the right project folder in PlatformIO

This repository holds the firmware for **several different boards**, each in its own subfolder. You must open the specific subfolder for the CYD 3.5” board — not the top-level `obd-esp32` folder — or PlatformIO won't know which firmware you mean.

1. In VS Code, go to the **File** menu and choose **Open Folder....**
2. Navigate into the project folder you got in step 3, then down into `boards` → `cyd-s3-3p5` → `firmware`.
3. Select the `firmware` folder itself (not a file inside it) and click **Open**.

You'll know you got it right if, after it opens, PlatformIO's icon in the left strip shows a project, and a file called `platformio.ini` is visible at the top level of the file list in the sidebar.

One important warning: that `platformio.ini` file pins an exact tool version (`platform = espressif32@7.1.3`). This is deliberate — a newer version is known to crash this firmware's Bluetooth code. If VS Code, PlatformIO, or anything else ever offers to “update” the platform, say no. Don't edit that line.

6. Format the microSD card

The board needs its microSD card formatted as **FAT32** — not exFAT, not NTFS, not “Mac OS Extended”. If the card isn't formatted this way, the board won't tell you — it'll just quietly log to its own tiny internal memory instead (room for a few minutes of driving), which defeats the point.

1. Put the microSD card into a card reader connected to your Mac.
2. Open **Disk Utility** (press **Cmd+Space**, type “Disk Utility”, press Return).
3. In the sidebar, click on the card itself (the physical device entry, not a volume indented underneath it).
4. Click **Erase** in the toolbar.
5. Give it a name if you like, and for **Format**, choose **MS-DOS (FAT)** — this is macOS's name for FAT32.
6. Click **Erase**, then **Done**.

If your card is larger than 32 GB, Disk Utility's GUI will grey out or simply not offer FAT32 as an option — this is a known macOS limitation, not a bug in the card. The simplest fix is to use a 32 GB or smaller card instead. If you specifically want to use a larger card, see the optional Terminal workaround at the bottom of this page.

Once formatted, insert the card into the board's SD slot before you power it on for the first time.

7. Connect the board

Plug the CYD 3.5" board into your Mac using a USB-C cable that you know carries data (not a charge-only cable — see the shopping list on the hub page).

What should happen: macOS may show a brief notification about a new USB device, and the board's screen should light up. This board uses the ESP32-S3 chip's own built-in USB connection — there's no separate USB-to-serial adapter chip to worry about, so on a modern Mac it should just work with no driver to install.

To check PlatformIO can see it: click the **PlatformIO** icon in the left strip, then look under the **PIO Home** → **Devices** section (or use the alien-head icon's quick-access menu) for a device list. You're looking for something named like `/dev/cu.usbmodemXXXX` (a string of numbers/letters after `usbmodem`). **I haven't personally verified this exact board model showing up this way on a brand-new Mac (unverified)** — if it doesn't appear, see the troubleshooting table below before assuming something is broken.

8. Build and upload the firmware

At the bottom of the VS Code window, once the `firmware` folder is open as your project, PlatformIO adds a thin blue toolbar with several small icons. The two you need:

- A **checkmark (✓)** icon — this **builds** the firmware (compiles the code without sending it anywhere yet).
- A **right-arrow (→)** icon — this **builds and uploads** (sends it to the board over USB). You can click this one directly; it builds first automatically.

1. Click the **right-arrow** icon.
2. A terminal panel opens at the bottom of the window and starts printing a lot of text — this is normal. **The first build can take several minutes** (it's compiling everything from scratch).
3. Watch for the end of the output:
 - A line reading **SUCCESS** (in green) means it built and uploaded.
 - Red text and a non-zero exit/error means it failed — see troubleshooting below.

9. Open the Serial Monitor

The “serial monitor” is just a window that shows text the board prints over the same USB cable while it's running — useful for confirming it booted correctly.

1. In the same bottom toolbar, click the **plug-shaped** icon.
2. A new terminal panel opens and starts printing the board's startup messages.

3. You should see boot text scroll past, and the board's screen should show its boot jingle / light up around the same time. Soon after, it should start scanning for the Bluetooth OBD adapter.

Opening the monitor resets the board and starts a fresh session, so this is a good moment to confirm things are alive before taking the board out to the car.

10. Troubleshooting (macOS-specific)

Symptom	Likely cause	Fix
No device/port appears in PlatformIO's device list	Charge-only USB-C cable	Swap to a cable you know transfers data (e.g. one you've used to sync a phone).
Still no port after swapping cables	Mac isn't seeing the hardware at all	Open Apple menu → About This Mac → More Info → System Information , then USB under Hardware. If the board doesn't appear there either, it's a hardware/cable/power problem, not software. (unverified exact wording of this menu may vary by macOS version)
Port appears but upload fails partway through	Board didn't enter its upload mode, or a loose connection	Unplug and replug the board, close any other app that might have the port open (another VS Code window, a serial terminal), and try the upload again.
"Permission denied" style error on the port	Uncommon on macOS compared to Linux, but (unverified) another process may be holding the port open	Quit other apps/terminals that might be using the port, replug the board, retry.
PlatformIO extension install hangs or fails	Slow/interrupted connection during the large first-time download	Check your internet connection and retry the Install button; if it partially installed, try Uninstall then Install again from the Extensions panel.

Symptom	Likely cause	Fix
Gatekeeper blocks VS Code or something else from opening	macOS's standard "downloaded from the internet" warning	Right-click the app, choose Open , then confirm in the dialog that appears (this remembers your choice after the first time).
A dialog about an unsigned/blocked system extension appears	This board shouldn't need one — it uses the chip's native USB connection, not a separate driver	(unverified) This almost certainly means something unrelated is happening. Don't grant the extension in System Settings → Privacy & Security just to make this board work; if you're stuck here, double-check you're using the right cable and port first.

What's next

Once the build says `SUCCESS` and the serial monitor shows the board booting, go back to the hub page's ["Checking it's actually working" section](#) for what you should see on the screen with the SD card in, and then with the car's Bluetooth adapter connected.

Optional, advanced: using a microSD card bigger than 32 GB

Skip this section entirely if you used a 32 GB or smaller card — it's not needed.

macOS's Disk Utility app refuses to offer FAT32 for cards above 32 GB through its normal window. There's a workaround, but it needs one Terminal command. If you've never used Terminal before, it's the plain black (or white) window app you can open from **Applications** → **Utilities** → **Terminal**, where you type commands instead of clicking buttons.

The command below erases the card and reformats it as FAT32 regardless of size.

This permanently deletes everything on the card — only run it on the card you intend to use for this project.

1. Open **Disk Utility**, click your card, and note its **identifier** at the bottom (something like `disk4`) — not a partition like `disk4s1`, the whole-disk one.
2. Open **Terminal** and type this, replacing `CARDNAME` with any name (letters/numbers, no spaces) and `diskN` with the identifier you noted (e.g. `disk4`):

```
diskutil eraseDisk MS-DOS CARDNAME diskN
```

This command tells macOS: “erase this entire disk and format it as MS-DOS/FAT32, naming it CARDNAME.” It’s the same operation Disk Utility’s GUI does for smaller cards — this is just the way to force it above the 32 GB limit.

3. Press Return, confirm the identifier is really your card (not your Mac’s own disk — double- and triple-check this), and let it finish.

If any of that is unclear, it’s genuinely simpler and safer to just use a 32 GB or smaller card instead.

Easy setup — Linux

New to this? Start at the [easy-setup hub page](#) for the shopping list and the `config.h` walkthrough.

This page covers only two things: installing the software on your Linux computer, and flashing the CYD 3.5” board. Everything else — what to buy, what to put in `config.h`, and how to tell the finished board is working — is on the hub page above.

A note on distros. The exact clicks below (software store names, app icons, where “Activities” is) are for **Ubuntu with the default GNOME desktop**, since that’s the most common beginner distro. If you’re on something else (Fedora, Mint, Pop!_OS, Arch-based, KDE instead of GNOME, etc.), the same steps exist but the app store and menus look different — we’ll call out the one place that’s genuinely different by distro family (the group name in the permissions step).

Almost everything here is point-and-click. There is exactly one place where you have to type a command into a terminal (a program where you type commands instead of clicking things) — step 8, for serial port permissions. We explain exactly what it does before you type it, and you’ll only ever need to do it once.

1. Install Visual Studio Code

VS Code is the program you’ll use to edit files and, with an add-on, build and flash the firmware.

On Ubuntu (or another GNOME-based distro): open your software store app — on Ubuntu this is **Ubuntu Software** (a shopping-bag icon in the dock/Activities), on other GNOME distros it may just be called **Software**. Search for “**Visual Studio Code**”, pick the one published by Microsoft, and click **Install**. Wait for it to finish, then you can open it from Activities the same way you’d open any other app.

If your distro doesn’t have VS Code in its software store (or you’d rather not use it), go to code.visualstudio.com in your browser and download the `.deb` package (Ubuntu, Mint, Pop!_OS and other Debian-based distros) or the `.rpm` package (Fedora and other Red Hat-based distros). Double-clicking the downloaded file normally opens your distro’s package installer with an **Install** button — click it.

Exact steps vary by distro and desktop environment; the above describes Ubuntu specifically.

2. Install the PlatformIO IDE extension

PlatformIO is the add-on that teaches VS Code how to build and upload this project's firmware.

1. Open VS Code.
2. Click the **Extensions** icon in the left-hand sidebar (it looks like four small squares, one pulled away from the others).
3. In the search box at the top, type **PlatformIO IDE**.
4. Click **Install** on the result published by PlatformIO.
5. Wait. The first install can take several minutes — it's downloading toolchains in the background, not just the extension itself. VS Code may ask to **reload** partway through; let it.

When it's done you'll see a new alien-head icon in the left sidebar — that's the PlatformIO home screen.

3. Get the project files

You need the `obd-esp32` project folder on your computer. Depending on how you were given access to it, do one of these:

If you have a repository URL (e.g. a link to a Gitea or GitHub-style address): in VS Code, open the **PlatformIO** home screen (the alien-head icon), and look for a **Clone / Open Project** option — or use VS Code's own **Source Control** icon in the sidebar, which has a **Clone Repository** button. Paste the URL in, and pick a folder to save it into (your home folder or **Documents** is fine).

If you were given a folder or a ZIP file directly (e.g. on a USB stick or downloaded link): if it's a ZIP, right-click it in your file manager (Files/Nautilus on Ubuntu) and choose **Extract Here** (or **Extract to...** and pick a destination). Put the extracted folder somewhere easy to find again, like your home folder or **Documents**. If it's already a plain folder, just copy it there the normal way.

Either way, make a note of where the `obd-esp32` folder ended up — you'll need to point VS Code at it in the next step.

4. Open the right project folder in PlatformIO

This repo holds firmware for **several different boards**, each in its own subfolder. You must open the one for the CYD 3.5" board specifically, not the repo's top-level folder — opening the wrong folder means PlatformIO won't find the right `platformio.ini` and the build step later will fail or build the wrong thing.

1. In VS Code, go to **File > Open Folder...**

2. Navigate into the `obd-esp32` folder you got in step 3, then into `boards/cyd-s3-3p5/firmware`.
3. Select that `firmware` folder (not any folder above or below it) and confirm.

VS Code will reload with that folder open, and after a moment PlatformIO should recognise it as a PlatformIO project (you'll see PlatformIO-specific icons appear in the bottom status bar).

5. Set up your WiFi and Telegram secrets

Before building, do the `config.h` step now. This is already fully covered on the hub page — see [Setting up your own secrets](#) — so we won't repeat it here. Come back to this page once that file exists.

6. Format the microSD card as FAT32

The board needs its microSD card formatted as **FAT32** (not exFAT, not NTFS) — see the hub page's [FAT32 warning](#) for why this matters (the board silently falls back to its own tiny internal memory if the card isn't right, with no warning on screen).

On Ubuntu/GNOME, the built-in **Disks** app does this without a terminal:

1. Put the microSD card in your computer (in a card slot, or a USB card reader if your computer doesn't have a slot).
2. Open **Disks** from Activities (search "Disks" — it's pre-installed on Ubuntu; the icon looks like a hard drive).
3. In the list on the left, select your SD card — be careful to pick the card itself, not your computer's own internal disk. Check the size shown matches your card.
4. Click the **gear/menu icon** below the volume diagram and choose **Format Partition...** (or **Format Disk...** if you want to wipe any existing partitions first).
5. In the dialog, set the type to **FAT32** (sometimes labelled "Compatible with all systems and devices (FAT)"), give it any name you like, and confirm. This will erase anything currently on the card.

Unlike Windows' and macOS's built-in formatting tools, Disks does **not** refuse to make a FAT32 partition on cards bigger than 32 GB, so this works regardless of your card's size.

If your distro doesn't have Disks installed, **GParted** is a similar point-and-click alternative — install it from your software store and use its **Format to > fat32** option on the card's partition.

Once formatted, insert the card into the board before you power it on for the first time.

7. Fix serial port permissions (one-time step)

This is the one unavoidable terminal step on Linux, and you'll only ever need to do it once on this computer.

Why this is needed: the CYD board uses the ESP32-S3 chip's own built-in USB connection — there's no separate USB-to-serial chip on this board. When you plug it in, Linux should show it as a device named something like `/dev/ttyACM0`. By default, though, your normal user account isn't allowed to read or write that device — only the system administrator (root) and members of a specific group are. If you skip this step, PlatformIO will fail partway through uploading with a “permission denied” style error, even though everything else is set up correctly. This is a permissions problem, not a missing driver — nothing extra needs installing for this board.

The fix is to add your user account to the group that's allowed to use serial devices, then make that change take effect.

1. **Open a terminal.** On Ubuntu, click **Activities** (top-left corner, or the Super/Windows key), type **Terminal**, and open the result — it's an app called **Terminal** with a dark window and a blinking cursor, already installed on Ubuntu.
2. Type this line exactly and press **Enter**:

```
sudo usermod -a -G dialout $USER
```

In plain English: this adds your user account to the `dialout` group, which is the group Linux uses to control who can open serial devices like `/dev/ttyACM0`. `sudo` means “do this as the administrator” — it will ask for your login password (the one you use to log into Ubuntu), which won't appear on screen as you type it; that's normal, just type it and press Enter.

If you're on an Arch-based distro (Arch, Manjaro, EndeavourOS), use `uucp` instead of `dialout`:

```
sudo usermod -a -G uucp $USER
```

3. **Log out and log back in, or restart your computer.** The group change doesn't take effect for your current login session — it only applies from your next login onward. This is the step people most often skip and then wonder why uploading still fails.

That's it — you won't need to touch a terminal again for the rest of this guide, or for any future rebuilds.

8. Connect the board

Plug the CYD board into your computer using a USB-C cable that actually carries data (see the warning on the hub page's shopping list — some cheap cables only charge). Use the port on the board itself, not on any adapter accessory it came with.

9. Build and upload the firmware

Back in VS Code, with the `boards/cyd-s3-3p5/firmware` folder open:

1. Look at the blue status bar along the **bottom** of the VS Code window — PlatformIO adds its own icons there.
2. Click the **checkmark icon** (✓) — this is **Build**. It compiles the firmware without uploading anything yet. A panel opens at the bottom showing progress.
3. Once Build finishes with no red errors, click the **right-arrow icon** (→) next to it — this is **Upload**. It builds again if needed and then sends the firmware to the board over USB.

How to tell it worked: the bottom panel ends with a green line saying something like
`===== [SUCCESS] =====`.

How to tell it failed: red text in that same panel. If the message mentions “Permission denied” on a port like `/dev/ttyACM0`, that's almost always step 7 — either you haven't logged out/in since running the command, or the command didn't run. Log out and back in and try Upload again.

The **first** build is the slow one — it has to compile everything from scratch and can take several minutes. Later builds, after small code changes, are much faster.

Do not let PlatformIO or VS Code “update” anything related to the `espressif32` platform version for this project if it offers to — this project deliberately pins `platform = espressif32@7.1.3` in `platformio.ini` because newer versions crash the Bluetooth library this firmware uses. If you're ever asked to update the platform, say no.

10. Watch it boot with the Serial Monitor

Still in the same bottom toolbar, click the **plug icon** — this opens the **Serial Monitor**, a live text view of whatever the board is printing as it runs.

Opening it resets the board, so you should see it boot from the top: lines about the screen initialising, the SD card mounting, and it starting to scan for your Bluetooth OBD adapter. You should also hear the board's short boot jingle and see its screen light up at the same time. This confirms the firmware is actually running, separately from what's shown on the board's own screen.

Troubleshooting (Linux-specific)

Problem	What to try
“Permission denied” on the port during Upload	You’re missing step 7, or you did it but haven’t logged out and back in yet (or restarted) since. Do that, then try Upload again.
No port appears at all / PlatformIO can’t find the board	Try a different USB cable (many are charge-only, not data). Try a different USB port. Make sure the board is actually powered — screen should light up. (unverified) Some distros run background services called ModemManager or brltty that are sometimes reported to interfere with serial devices like this one; this is general community knowledge, not something confirmed on this specific project, so try the cable/port checks first.
Upload starts but fails partway through	Try again — sometimes a stale connection from a previous attempt causes this. If it keeps happening, unplug and replug the board and retry.
PlatformIO IDE extension install hangs or fails in VS Code	Check your internet connection — the first install downloads toolchains, which can be large. Close and reopen VS Code and try again; if it’s still stuck, a full restart of your computer and reinstalling the extension has fixed similar extension issues in other projects (unverified for this specific extension) .

What’s next

Once Upload succeeds and the Serial Monitor shows the board alive, go back to the hub page’s [Checking it’s actually working](#) section — it walks through what the screen should show once you plug the board into your car.